PHIẾU THỰC HÀNH — 09/10/2026

Họ tên: Lê Hoàng Đức Thịnh MSSV: N24DCDT088 Lớp: D24CQDT01-N

GitHub:
https://github.com/N24DCDT088LeHoangDucThinh/N24DCDT088_ThucHanh_Julia
Commit SHA: 2d5af8903441430ab171e6aa57ddd4082abca0fe

1. Mô tả một luồng ứng dụng

Ghi mục tiêu, đầu vào, đầu ra và sơ đồ truyền dữ liệu giữa các chức năng.

Mục tiêu: chọn các đơn có tổng lợi ích lớn nhất trong ngân sách B, mỗi đơn chỉ chọn một lần. Đầu vào: don_hang.csv (ID, điểm giao, phục vụ, lợi ích), ban_do.csv (cạnh u,v) và B = 8 của MINI001. Đầu ra: tập chọn, tuyến, chi phí, trạng thái giao và ket_qua.csv.
Luồng: đọc CSV → sắp danh mục theo ID/tra cứu → BFS từ kho 1 → tính chi phí τ = 2h + s → sắp ưu tiên và chọn bằng DP → FIFO giao đơn/ngăn xếp hoàn tác → xuất CSV. BST tra cứu cùng danh mục; danh sách liên kết giữ kế hoạch.
Phạm vi phiếu: truy vết MINI001 và kết quả chạy thực tế trên Windows 11, Julia 1.12.7. Đã hoàn thiện bốn hàm, chạy các pha kiểm thử và xuất CSV; dữ liệu cá nhân dùng MSSV N24DCDT088.

2. Bảng truy vết đường và thời gian

ID	Đường lượt đi	Số cạnh h	2h + phục vụ
1	1 → 2	1	2 × 1 + 1 = 3
2	1 → 2 → 4	2	2 × 2 + 1 = 5
3	1 → 2 → 4 → 6	3	2 × 3 + 1 = 7

3. Minh chứng bốn hàm

NV1 · Hoàn thiện merge_sort(a, before)

Ý tưởng/điều kiện áp dụng: Chia dãy thành hai nửa, đệ quy sắp từng nửa rồi trộn. Dừng khi có không quá một phần tử và trả bản sao. Khi hai khóa hòa, lấy phần tử bên trái trước; chép đủ phần dư. Comparator phải nhất quán, không dùng sort/sort! và không sửa đầu vào. Truy vết: [3,1,2] → [3] và [1,2] → trộn thành [1,2,3].

Lời gọi và kết quả kiểm thử: merge_sort([3,1,2], (<)); kỳ vọng [1,2,3]. Kiểm tra thêm dãy rỗng, một phần tử, khóa trùng, comparator giảm và tính ổn định. Lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=sort. Thực tế: đạt 8/8 phép kiểm tra.

Chi phí thời gian/bộ nhớ: O(n log n) thời gian. Hiện thực đệ quy bằng lát cắt và tạo dãy kết quả mới ở mỗi lần trộn; bộ nhớ sống đồng thời O(n), ngăn xếp O(log n), tổng lượng cấp phát qua các mức O(n log n).

NV2 · Hoàn thiện binary_search(a::Vector{Order}, id::Int)

Ý tưởng/điều kiện áp dụng: Dãy Order phải tăng theo ID, các ID duy nhất. Đặt left = 1, right = n; khi left ≤ right, tính mid = left + (right − left) ÷ 2. So sánh a[mid].id; khớp thì trả Order, nhỏ hơn thì left = mid + 1, lớn hơn thì right = mid − 1. Hết khoảng trả nothing; không thay dữ liệu.

Lời gọi và kết quả kiểm thử: binary_search(by_id,2) kỳ vọng Order(2,4,1,8); ID 99 và dãy rỗng trả nothing. Truy vết ID 2 trên [1,2,3]: left = 1, mid = 2, right = 3; khóa giữa = 2 → tìm thấy. Kiểm tra thêm ID đầu/cuối. Lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=search. Thực tế: đạt 5/5 phép kiểm tra.

Chi phí thời gian/bộ nhớ: O(log n) thời gian, O(1) bộ nhớ phụ với vòng lặp. Sắp danh mục ban đầu tốn O(n log n), không cần sắp lại sau mỗi lần tìm nếu dữ liệu không đổi.

NV3 · Hoàn thiện bfs_routes(graph, source=1)

Ý tưởng/điều kiện áp dụng: Đồ thị vô hướng không trọng số, mỗi cạnh mất một đơn vị thời gian. Dùng CircularQueue; đặt distance = −1, nguồn có distance = 0 và parent = 0. Khi phát hiện v từ u, ghi distance[v] = distance[u] + 1 và parent[v] = u trước khi đưa v vào FIFO. Lần ngược parent rồi đảo để dựng tuyến; đỉnh không tới được giữ −1 và tuyến rỗng. Trả distance, parent, routes.

Lời gọi và kết quả kiểm thử: bfs_routes(graph,1) kỳ vọng distance = [0,1,1,2,2,3]. FIFO lần lượt: [1] → [2,3] → [3,4] → [4,5] → [5,6] → [6] → []. Tính costs[o.id] = 2 × distance[o.destination] + o.service. Kiểm tra thêm nguồn 4 và đỉnh cô lập; chấp nhận các tuyến ngắn nhất khác nhau. Lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=bfs. Thực tế: đạt 7/7 phép kiểm tra.

Chi phí thời gian/bộ nhớ: BFS cốt lõi O(V + E) thời gian, O(V) bộ nhớ phụ ngoài đồ thị. Nếu dựng/lưu mọi routes, cộng O(L) thời gian và bộ nhớ, với L là tổng số đỉnh trên các tuyến; L có thể $O(V^2)$. Tính chi phí n đơn thêm O(n).

NV4 · Hoàn thiện knapsack(orders, costs, budget)

Ý tưởng/điều kiện áp dụng: Ngân sách B nguyên không âm; chi phí tra theo ID từ BFS. F(i,b) là lợi ích tốt nhất khi xét i đơn đầu với ngân sách b. Nếu $\tau_i > b$ thì giữ F(i−1,b); ngược lại lấy max(F(i−1,b), $lợi_ích_i$ + F(i−1,b−τ_i)). F(0,b) = F(i,0) = 0 vì chi phí đơn dương. Dùng table[i+1,b+1]; truy vết từ (n,B), giảm i mỗi bước. Khi hòa giữ phương án không lấy. Trả ids, value, cost, table.

Lời gọi và kết quả kiểm thử: knapsack(orders,costs,8) kỳ vọng ids có tập {1,2}, value = 14, cost = 8, bảng 4 × 9. Kiểm tra B = 0, dãy rỗng, ngân sách âm phải báo lỗi, không lặp ID và giá trị bằng vét cạn. Lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=dp. Thực tế: đạt 12/12 phép kiểm tra.

Chi phí thời gian/bộ nhớ: Bảng đầy đủ O(nB) thời gian và O(nB) bộ nhớ; truy vết O(n). Đây là độ phức tạp theo giá trị B. Vét cạn tính tổng từng tập cần O(n × 2^n), chỉ phù hợp bộ nhỏ.

4. Quyết định chọn đơn

Điền ngân sách, tập chọn, tổng thời gian/lợi ích. Giải thích một ô DP và một bước truy vết; đối chiếu với vét cạn.

Tính tay: B = 8; chọn ID {1,2}; tổng thời gian 3 + 5 = 8, lợi ích 6 + 8 = 14. F(2,8) = max(F(1,8), 8 + F(1,3)) = max(6,14) = 14, nằm tại table[3,9]. F(3,8) = max(14, 10 + F(2,1)) = 14.
Truy vết: tại (3,8) bỏ ID 3; tại (2,8) lấy ID 2, còn b = 3; tại (1,3) lấy ID 1, còn b = 0.
Đối chiếu tính tay 8 tập con (chi phí; lợi ích): ∅ (0;0), {1} (3;6), {2} (5;8), {3} (7;10), {1,2} (8;14), {1,3} (10;16), {2,3} (12;18), {1,2,3} (15;24). Ba tập cuối vượt B; {1,2} tối ưu trong các tập hợp lệ. Chạy --mini cho kết quả DP = 14 và vét cạn = 14; tập chọn [1,2], tổng chi phí 8.

5. Giao và hoàn tác

Trạng thái	Queue	Stack	Đã dùng	Đã nhận
Sau lập kế hoạch	[1,2]	[]	0	0
Sau giao một đơn	[2]	[1]	3	6
Sau hoàn tác	[1,2]	[]	0	0

6. Đầu ra và nhận xét

CSV đã xuất: data/MINI001/ket_qua.csv và data/N24DCDT088/ket_qua.csv. Với MINI001, ID 1,2 được chọn và đã giao; ID 3 không chọn.
Notebook TH_2026_10_09_Pluto.jl gọi cùng mã lõi; READ_DATA và RUN_APP đã đặt true. Các ô thực hiện lập kế hoạch, giao một đơn, hoàn tác, giao hết và kiểm tra bất biến. Mã nguồn cùng kết quả kiểm thử nằm trong kho GitHub ở đầu phiếu.
CSV cá nhân N24DCDT088 có 8 đơn; các ID 1,3,4,5,6,7 được chọn và đã giao, tổng chi phí 24, tổng lợi ích 71. ID 2 và 8 không chọn. Đây là kết quả riêng của Thịnh.

Lỗi đã phát hiện và cách sửa: Khung ban đầu còn bốn hàm TODO gọi error("CHUA_HOAN_THANH..."). Đã thay các phần này bằng Merge Sort ổn định, tìm kiếm nhị phân, BFS dùng FIFO và DP ba lô 0/1. Kiểm riêng từng pha trước khi chạy toàn bộ; file ket_qua_kiem_thu.txt ghi tất cả các pha đều đạt.

Một chức năng thay đổi ảnh hưởng đến chức năng sau như thế nào? Nếu bỏ hai cạnh (2,4), (3,4), đường ngắn nhất tới 4 dài 4 cạnh, nên chi phí ID 2 tăng từ 5 lên 9. Khi B = 8, DP chuyển từ {1,2} sang {3}, lợi ích 10; FIFO và CSV cũng phải đổi. Đây là tình huống đối chiếu trong pha app.
Kết quả thực tế: data 12/12, sort 8/8, search 5/5, bfs 7/7, dp 12/12, app 16/16; --phase=all đạt 60/60 phép kiểm tra. Minh chứng: ket_qua_kiem_thu.txt. Lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=all.

Câu hỏi cuối buổi

Q1. Vì sao không được dùng dãy đã sắp giảm theo lợi ích/τ để tìm kiếm nhị phân theo ID? Nêu tiền điều kiện của hàm tìm kiếm.

Dãy giảm theo lợi ích/τ không bảo đảm ID tăng, nên loại nửa khoảng dựa trên ID có thể bỏ mất đơn cần tìm. Tiền điều kiện: danh mục sắp tăng theo Order.id và các ID duy nhất. Phải giữ danh mục ID riêng với dãy ưu tiên. Bộ mini tình cờ cùng thứ tự không chứng minh hai cách sắp tương đương.

Q2. Với đơn ID 2 tới điểm 4, phục vụ 1 và bản đồ mẫu, giải thích từng thành phần của thời gian thực hiện τ. BFS có tiếp tục cho thời gian ngắn nhất nếu các cạnh mang thời gian khác nhau không?

ID 2 giao tới đỉnh 4: tuyến $1 \to 2 \to 4$ có h = 2 cạnh; đi mất 2, về mất 2, phục vụ mất 1. Vậy $\tau = 2 \times 2 + 1 = 5$ đơn vị thời gian. BFS tối thiểu số cạnh; khi thời gian các cạnh khác nhau, tuyến ít cạnh nhất không nhất thiết nhanh nhất. Với trọng số không âm, có thể dùng Dijkstra để tìm tổng thời gian nhỏ nhất.

Q3. Giải thích hai phương án tạo ô F(2,8) và vị trí của ô đó trong table Julia. Vì sao nhánh lấy đơn phải dùng hàng trước?

Không lấy ID 2: F(1,8) = 6. Lấy ID 2: lợi ích 8 cộng F(1,8−5) = 6, tổng 14. Do đó F(2,8) = 14, lưu tại table[3,9]; hai ô tham chiếu là table[2,9] và table[2,4]. Nhánh lấy dùng hàng trước để chỉ xét các đơn trước ID 2, tránh chọn lại cùng đơn và biến bài toán thành ba lô không giới hạn.

Q4. Sau khi giao ID 1 rồi hoàn tác ngay, FIFO, ngăn xếp, tổng thời gian và tổng lợi ích phải thay đổi thế nào? Nêu một bất biến kiểm tra việc không mất hoặc lặp đơn.

Ban đầu FIFO [1,2], Stack [], đã dùng/đã nhận = 0/0. Giao ID 1: FIFO [2], Stack [1], hai tổng = 3/6. Hoàn tác: lấy ID 1 khỏi đỉnh Stack, trả về đầu FIFO → [1,2], Stack [], hai tổng = 0/0. Bất biến: các ID chờ và đã giao không trùng nhau, không lặp trong mỗi cấu trúc; hợp của chúng bằng tập chọn {1,2}. Các tổng phải bằng tổng chi phí/lợi ích của các đơn đã giao.

Q5. Cho biết độ phức tạp theo n,V,E,B của sắp xếp trộn, tìm kiếm nhị phân, BFS cốt lõi và DP bảng đầy đủ. Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n)?

Với n đơn, V đỉnh, E cạnh và ngân sách B: Merge Sort O(n log n) thời gian; tìm kiếm nhị phân O(log n); BFS cốt lõi O(V + E); DP bảng đầy đủ O(nB) thời gian và bộ nhớ. BFS dùng O(V) bộ nhớ phụ, còn lưu mọi tuyến có thể cần $O(V^2)$.
BST tra cứu O(h), h là chiều cao cây. Cây cân bằng có h = O(log n), nhưng BST thông thường có thể lệch thành chuỗi, h = O(n), nên không thể khẳng định mọi BST đều tra cứu O(log n).